

Mission-DT

User and Reference Manual

A Mission-Level Digital Twin for Hybrid Fleets of Physical and Virtual Unmanned Vehicles

Prof. Filipo Novo Mór

PUCRS — School of Technology, Porto Alegre, Brazil

github.com/ProfessorFilipo/mission-dt

Revised edition — reflects the repository as of this printing

Table of Contents

(If the entries below do not appear, right-click anywhere in this area and choose "Update Field" — or "Update Table of Contents" on Mac — once the document is open in Word.)

Table of Contents	2
1. Overview	4
1.1 Hybrid agents	4
1.2 Relationship to Fleet-DT and the paper.....	4
2. Architecture.....	4
2.1 Mission core (mission_dt/core.py)	4
2.2 Physical and virtual agents (mission_dt/agents.py).....	5
2.3 MQTT topic contract.....	5
2.4 The 3D mission view (viz/mission_viz.py).....	5
2.5 Package-level fix (mission_dt/__init__.py).....	5
3. Installation.....	6
3.1 Common steps (all platforms)	6
3.2 Linux	6
3.3 macOS.....	6
3.4 Windows 10/11	6
4. Quick Start	7
4.1 One-command launch	7
4.2 Manual, step by step.....	7
5. Mission Configuration.....	8
5.1 Schema	8
5.2 Auto-generated profiles.....	8
5.3 Bundled example configurations.....	8
6. Staging a Deterministic Screenshot.....	9
6.1 How it works	9
6.2 The bundled composition	9
6.3 Usage.....	9
6.4 Adjusting the composition.....	9
7. The 3D Mission View	11
7.1 Controls.....	11
7.2 Vehicle models and status	11
7.3 Safety spheres.....	11
7.4 Checkpoints.....	11
7.5 Recording	12
8. The Control Panel	13
9. Reproducing the Experiments	14
9.0 One-command battery	14
9.1 E1 — Real-time scalability	14
9.2 E2 — Bandwidth regulators	14

9.3 E3 — Swarm-reaction propagation	14
9.4 E4 — Twin fidelity under packet loss	15
9.5 Cross-platform reproduction	15
10. Hands-on Test Scenarios	16
T1 — One-command launch	16
T2 — Manual four-terminal quick start	16
T3 — Explicit checkpoints.....	16
T4 — Explorador profile	16
T5 — Deterministic staged scene.....	16
T6 — Recording.....	16
T7 — Live monitoring	16
T8 — Ghost cleanup	16
T9 — Quick experiment sanity check	16
T10 — Checkout verification.....	16
11. Troubleshooting	18
"Ghost" agents appear in the 3D view or panel.....	18
macOS: "too many open files" with large fleets.....	18
Windows: PowerShell refuses to activate the virtual environment	18
Windows: broker connection fails even though Mosquitto is "running"	18
Windows (Git Bash): "python: command not found" after activating a venv	18
A vehicle's heading slowly rotates even though it should be stationary.....	18
Two very close vehicles look like a single fused shape	18
12. Repository Reference	19

1. Overview

Mission-DT is a mission-level Digital Twin (DT) architecture for hybrid fleets of unmanned aerial and surface vehicles. Unlike vehicle-level or fleet-level twins, Mission-DT promotes the mission itself to a first-class twinned entity, formally:

```
M = < states, transition functions, mission goals g, context  $\phi$  >
```

The mission context ϕ is operational, not merely descriptive: at every frame the core evaluates pairwise inter-agent distances and can override an agent's own controller with corrective actuation whenever two agents of the same domain (aerial or surface) come within a configurable separation threshold. This is the mechanism through which one agent's manoeuvre propagates to its neighbours as mission-level coordination.

1.1 Hybrid agents

Every agent is heterogeneous (domain: aerial or surface, each with its own transition and actuation model) and hybrid (kind: physical or virtual). A physical agent is a real vehicle whose autopilot telemetry is bridged into MQTT (typically via ArduPilot and a MAVLink-to-MQTT bridge). A virtual agent is a vehicle-level software emulator — itself an independent digital twin, with its own kinematic model, state, and MQTT connection — that publishes and consumes exactly the same telemetry and actuation topics as a physical vehicle. The mission core cannot tell the two apart, which is what enables mixed-reality operation: a mission can mix real and simulated vehicles transparently, and either can replace the other without touching the core.

1.2 Relationship to Fleet-DT and the paper

Mission-DT builds on the group's earlier Fleet-DT work (a fleet-level, homogeneous-vessel digital twin) and lifts the abstraction to the mission level, adding heterogeneous domains, hybrid agent kinds, and mission-level swarm coordination. This manual documents the reference implementation exactly as published in the repository; all quantitative results referenced here (Sections 9–10) come from the raw measurement files under `results/*.json`, which are the single source of truth used to write the paper.

2. Architecture

The implementation is a lightweight set of decoupled Python components communicating exclusively over MQTT — no component ever calls another directly. Three principles run through the design: uniform message contracts for physical and virtual agents, temporal decoupling between each agent's native sensor rate and the fixed Digital Twin frame rate, and process-level isolation between the mission core, every agent, and the visualization frontend.

2.1 Mission core (`mission_dt/core.py`)

The `MissionDT` class is the real-time component that maintains the online state of the mission twin. It subscribes to `missiondt/agents/+/telemetry` and `missiondt/agents/+/register`, and keeps one `AgentRecord` per agent — current state, a bounded history queue (used for dead-reckoning), the current goal, and a last-seen sequence number.

The core runs a fixed-period frame loop (default 125 ms, i.e. 8 Hz). Once per frame it:

- consumes the newest pending telemetry message per agent, discarding older ones by sequence number (a late packet can never overwrite a newer state within the same frame);
- dead-reckons an agent's state from its own recent history if no fresh telemetry arrived that frame (only once the agent has been seeded by at least one real message);
- computes actuation via a goal-seeking controller (bearing-and-distance based throttle/steer, plus climb for aerial agents);
- when `swarm=True`, evaluates the mission context ϕ — pairwise same-domain distances — and, for any agent within `sep_m` (default 12 m) of its nearest same-domain neighbour, overrides its steer command to point away from that neighbour, caps its throttle, and flags the actuation message `avoid=True`;
- publishes the resulting actuation to `missiondt/agents/<id>/actuation`.

Every frame's compute time, every stale/duplicate update, every avoidance event, and every message's end-to-end latency are recorded internally — this is what the experiment scripts in Section 9 read out.

2.2 Physical and virtual agents (mission_dt/agents.py)

A VirtualAgent is a Python thread that owns its own MQTT connection. On start it publishes a retained registration message (domain and kind) to missiondt/agents/<id>/register, subscribes to its own actuation topic, and begins an internal simulation loop at its native sensor rate (50 Hz by default). A configurable bandwidth regulator decimates the outbound telemetry rate to the DT's own 8 Hz frame rate before publishing — this is the same mechanism a physical vehicle's own telemetry pipeline would apply, and it is what Section 9.2 (E2) measures.

A virtual agent can additionally simulate imperfect conditions for experimentation: jitter (Gaussian sensor noise, on by default) and loss (an independent packet-drop probability applied at the regulator's output, used by the E4 fidelity experiment). On a clean exit, the agent clears its own retained registration (publishes an empty retained payload) so no "ghost" agent lingers in the broker after the process ends.

2.3 MQTT topic contract

Every physical or virtual agent, and the mission core, communicate exclusively through the following topics. This is the complete contract — implementing it (in any language, on any device) is sufficient to appear as a first-class hybrid agent to the mission core.

Topic	Direction	QoS / retain	Payload (summary)
missiondt/agents/<id>/telemetry	agent → core	0 / no	gps [lat,lon,alt], att [r,p,yaw], vel [u,v,w], imu, mag, baro, battery, seq, t_pub
missiondt/agents/<id>/actuation	core → agent	0 / no	tau (throttle), alpha (steer), climb (aerial only), avoid flag, t_pub, trig_id/trig_t when avoiding
missiondt/agents/<id>/register	agent → core	1 / yes	domain, kind — empty payload on clean exit clears the retained registration
missiondt/mission/checkpoints	mission → view	1 / yes	{name: [lat, lon, alt_m]} — labels the 3D view; alt>0 air corridor, 0 surface, <0 submerged
missiondt/mission/routes	mission → view	1 / yes	{agent_id: [[lat,lon,alt_m], ...]} — drawn as a closed green line in the 3D view when ROTA is enabled

QoS 0 (telemetry, actuation) is a deliberate choice: it removes acknowledgement/retransmission overhead from the real-time path, so a slow or disconnected subscriber (such as a lagging 3D view) can never back-pressure the mission core or an agent. QoS 1 with retain=true (registration, checkpoints, routes) is used only for state that a late-joining subscriber — the 3D view started after the mission, for instance — must still be able to recover.

2.4 The 3D mission view (viz/mission_viz.py)

The visualizer is intentionally outside the real-time control path: it runs as a separate operating-system process with its own MQTT session, subscribing independently to telemetry, registration, checkpoints, and routes. Telemetry ingestion happens on the MQTT client's own network thread; rendering happens on the game engine's own loop. A slow GPU or a lagging visualizer therefore only ever affects its own frame rate — never the mission core's frame timing. It can be started, stopped, or replicated at will without touching the mission.

2.5 Package-level fix (mission_dt/__init__.py)

On POSIX systems (Linux, macOS), importing the mission_dt package raises the process's open-file-descriptor limit to 4096 if the platform default is lower — macOS ships a default soft limit of 256, which is easily exhausted once a fleet of 100 or more agents is each holding open several sockets. This is a no-op on Windows.

3. Installation

Requirements: Python 3.10 or later, and a Mosquitto MQTT broker reachable at 127.0.0.1:1883. The 3D view additionally needs a machine with a display and a GPU.

3.1 Common steps (all platforms)

```
python3 -m venv .venv
# activate it (see your platform below), then:
pip install -r requirements.txt --break-system-packages
# 3D view only, on the machine with a display/GPU:
pip install ursina imageio imageio-ffmpeg
# optional prettier terminal panel:
pip install rich
```

3.2 Linux

```
sudo apt install mosquitto mosquitto-clients
source .venv/bin/activate
```

3.3 macOS

```
brew install mosquitto
brew services start mosquitto # or: mosquitto -v
source .venv/bin/activate
```

macOS's default shell file-descriptor limit (256) is too low for large fleets. `mission_dt/__init__.py` raises it automatically to 4096 on import — no manual step needed.

3.4 Windows 10/11

```
python -m venv .venv
.venv\Scripts\Activate.ps1
# Mosquitto: install from mosquitto.org, or
choco install mosquitto
```

Two Windows-specific notes worth knowing before you hit them:

- PowerShell's default execution policy can refuse to run `.venv\Scripts\Activate.ps1` ("not digitally signed"). Fix for the current session only: `Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass`, then activate again.
- If Mosquitto was installed as a Windows service, it is likely already running in the background at all times (check with `Get-Service -Name "*mosquitto*"`) — you do not need to start it manually, and starting a second copy will conflict on the port.

4. Quick Start

4.1 One-command launch

run_mission.sh checks your virtual environment (Unix .venv/bin or Windows .venv/Scripts, or an already-active one), starts Mosquitto if it is not already running, clears leftover retained MQTT state, launches the mission core in the background, and opens the 3D view in the foreground:

```
./run_mission.sh                # default mission  
./run_mission.sh configs/mission_photo.json # any other mission file
```

Press ESC in the 3D view: this also stops the background mission core cleanly.

4.2 Manual, step by step

Useful when you also want the terminal panel running, or want to see each component's own log output:

```
mosquitto -v                # terminal 1: broker  
python experiments/demo_mission.py    # terminal 2: mission  
python viz/mission_viz.py    # terminal 3: 3D view  
python experiments/panel.py    # terminal 4: dashboard
```

5. Mission Configuration

experiments/demo_mission.py reads a JSON configuration file (--config, default configs/mission_default.json) describing the fleet and its routes.

5.1 Schema

Field	Type	Meaning
drones	object	{"aerial": N, "surface": M} — fleet composition
profile	string	"orbitas" "paralelo" "explorador" "aleatorio" "checkpoints"
radius_m	number	Reference radius (metres) for the auto-generated profiles
arrive_m	number	Distance (metres) at which an agent is considered to have reached a waypoint
aerial_alt_m	number	Cruise altitude (metres) assigned to aerial agents by the auto-generated profiles
checkpoints	object	{"P01": [lat, lon, alt_m], ...} — alt>0 air corridor, 0 surface, <0 submerged
assignments	object	{"sur00": ["P01", "P02", ...], ...} — explicit per-agent route; a checkpoint may appear in more than one agent's list

5.2 Auto-generated profiles

Profile	Behaviour
orbitas	Antipodal patrol across a circle of radius_m — every agent's route crosses the centre, producing continuous separation-rule activity.
paralelo	Parallel lanes; each agent shuttles back and forth along its own lane.
explorador	Surface vessels advance in line (parallel lanes); each paired aerial agent's goal is a point 25 m ahead of its vessel, continuously recomputed from the vessel's live heading.
aleatorio	Each agent gets three random waypoints converging on one shared destination; a new random route is generated every time an agent completes a lap.

When assignments is present, it always takes precedence over the profile field for the agents it lists; any agent not covered by assignments falls back to holding its starting position.

5.3 Bundled example configurations

- configs/mission_default.json — 5 aerial + 5 surface, orbitas profile, no explicit checkpoints.
- configs/mission_checkpoints_example.json — 2 aerial + 3 surface, five checkpoints (P01–P05, including one submerged), a small hand-written explicit-route example.
- configs/mission_photo.json — 3 aerial + 3 surface, ten checkpoints (P01–P09 and P13, all three media represented), a denser composition — this is the configuration used to compose the paper's Figure 7 via a live, non-frozen mission.

Checkpoints and computed routes are published retained on missiondt/mission/checkpoints and missiondt/mission/routes, so the 3D view can label them and draw them even if it is started after the mission — no configuration file of its own.

6. Staging a Deterministic Screenshot

Composing a good illustrative screenshot of a live mission (for a paper, a slide deck, or documentation) by watching a running simulation and waiting for an interesting moment is slow, repetitive, and unreliable — the moment you want may last only a second or two. `experiments/staged_photo.py` takes a different approach: it places every agent directly at a chosen position and heading and holds it there indefinitely, so the desired composition is present and stable from the very first frame.

6.1 How it works

Each frozen agent is given a goal equal to its own current position, offset by one metre along its intended heading (not exactly at its own position — a goal identical to the current position makes the bearing calculation degenerate, which lets the steering controller slowly rotate the vehicle toward a meaningless heading over time even though it never translates). The one-metre offset keeps the goal-seeking controller's throttle at zero — the distance is below the controller's own arrival threshold — while still giving it a well-defined, unmoving bearing to settle onto and hold.

The mission core in this script runs with `swarm=False`. This is deliberate: the safety spheres shown in the 3D view are computed purely from same-domain distance on the client side (`viz/mission_viz.py`), independently of whether the core's own separation override is active — so the spheres are exactly as correct either way. Leaving the override on would continuously steer the frozen agents' heading away from their nearest same-domain neighbour, fighting the very heading this script sets up.

6.2 The bundled composition

The script freezes eight agents and lets a ninth move for real, arranged as a compact chain so no part of the frame is empty background:

Agents	Configuration	Resulting state
sur00, sur01	Surface pair, ~11 m apart, offset laterally and angled toward each other (not perfectly co-linear, to reduce hull-model overlap)	Orange conflict spheres
aer00, aer01	Aerial pair, ~18 m apart, hovering above the same cluster	White approach spheres
sur03, sur04	Surface pair, ~2 m apart, perpendicular headings	Red near-collision spheres
sur02, aer02	Parked well clear of every other cluster	No spheres — normal operation, for contrast
sur05	A real, moving agent patrolling a loop around the whole composition	Demonstrates the ROTA route-line feature live

All cross-cluster, same-domain distances are kept above the 24 m visibility threshold, so the three clusters read as visually and functionally distinct — no sphere bleeds from one group into another.

6.3 Usage

```
python experiments/staged_photo.py    # terminal 1 – scene freezes instantly
python viz/mission_viz.py             # terminal 2 – screenshot whenever you like
```

The eight frozen agents are stable from the first rendered frame; there is no need to wait. The ninth agent (sur05) keeps moving in the background for as long as the script runs — wait for it to be in an unobtrusive part of its loop if you want it out of the frame, or leave it in for a touch of life.

6.4 Adjusting the composition

All coordinates are given as simple metre offsets from named anchor points near the top of the file (`MAIN_E/MAIN_N`, `BRIDGE_E/BRIDGE_N`, `CALM_E/CALM_N`), so the whole composition can be rearranged without touching the freezing logic itself. Two physical limits are worth keeping in mind when editing distances: the surface-vessel hull model is roughly 12 m long and 5 m wide at the visualizer's default scale, so pairs placed near the orange threshold (12 m) or especially the red threshold (3 m) will show some unavoidable visual overlap — orienting a close pair with parallel or

perpendicular headings (rather than perfectly co-linear, nose-to-nose) noticeably improves how distinguishable the two hulls remain at close range.

7. The 3D Mission View

`viz/mission_viz.py` is a standalone Ursina/Panda3D application. It must run on a machine with a display and a GPU; it connects to the broker as an independent MQTT client and can be started, stopped, or run from a different machine than the mission core, without any coordination beyond the broker address.

```
python viz/mission_viz.py --host 127.0.0.1
```

7.1 Controls

Button / key	Action
FOTO / P	Screenshot → captures/
REC / R	Start or stop MP4 recording (H.264, encoded via imageio-ffmpeg) → captures/
HQ / H	Toggle 30 fps high-quality vs 15 fps low-quality recording (cannot be changed mid-recording)
ID / I	Toggle drone name labels
ROTA / K	Toggle planned-route lines (green, closed loop); grey dots always show the travelled trail
LEG / L	Toggle a translucent on-screen legend
PANEL / O	Toggle a floating agent table inside the 3D window (reuses the same Watch class as the terminal panel)
G	Toggle the sky/grid backdrop
ESC	Quit (stops any in-progress recording first)

Mouse: right-drag to orbit, scroll to zoom, middle-drag to pan (standard Ursina editor camera).

7.2 Vehicle models and status

- Aerial agents render as an orange quadcopter (four arms, four rotors, a white nose marker for heading) with a vertical altitude reference line.
- Surface agents render as a blue vessel hull with a bow and a white tip marker for heading.
- An agent that has not published telemetry for more than 1.5 s turns solid grey.
- An agent whose reported battery voltage drops below 17.6 V shows a pulsing red marker above it.

7.3 Safety spheres

Computed independently for every agent from the distance to its nearest same-domain neighbour (aerial agents are only compared against other aerial agents; surface against surface):

Neighbour distance	Sphere
≥ 24 m	Hidden
12 m – 24 m	White (approaching)
3 m – 12 m	Orange (conflict — matches the mission core's default separation threshold)
< 3 m	Red (near-collision)

7.4 Checkpoints

Labelled pins appear automatically whenever the mission publishes to `missiondt/mission/checkpoints` — no local configuration needed. Colour encodes the medium:

- Cyan — surface checkpoint (altitude = 0).
- Orange — air-corridor checkpoint (altitude > 0), shown at its true height with a vertical reference line.
- Violet — submerged checkpoint (altitude < 0), labelled with its depth in metres.

7.5 Recording

REC captures raw frames via Panda3D's own movie-capture facility and, on stop, encodes them into an H.264 MP4 (ready for direct upload to YouTube or Vimeo) using imageio-ffmpeg, then deletes the intermediate frame images. If imageio is not installed, the raw frames are kept on disk along with the equivalent manual ffmpeg command to encode them yourself.

8. The Control Panel

experiments/panel.py is a standalone terminal dashboard: an independent MQTT client (like the 3D view) that subscribes to every agent's telemetry and registration topics and prints one row per agent, refreshed twice a second.

```
python experiments/panel.py --host 127.0.0.1
```

Column	Meaning
ID / DOMAIN / KIND	Agent identifier, aerial/surface, physical/virtual
LAT / LON / ALT m	Current reported position
SPD	Current forward speed (m/s)
VBAT	Current battery voltage
Hz	Observed telemetry rate for this agent
AGE s	Seconds since the last telemetry message
STATUS	OK, STALE (>1.5 s since last message), or LOWBATT (<17.6 V)

If the rich package is installed, the table renders live and colour-coded (green = OK, grey = STALE, red = LOWBATT) via rich.live.Live; otherwise the panel falls back automatically to plain, periodically reprinted rows — no functionality is lost, only the presentation. The same Watch class backs the in-3D-view PANEL toggle described in Section 7.

9. Reproducing the Experiments

All four experiments write raw measurements to `results/*.json`; every number quoted in the paper is read directly from those files. Unless noted otherwise, figures below were measured on a deliberately constrained platform: one virtual CPU (Xeon, 2.10 GHz), 4 GiB RAM, Linux, Mosquitto 2.x — so the mission core is competing for CPU time with the entire emulated fleet.

9.0 One-command battery

`run_all.sh` runs the entire E1–E4 battery plus figure regeneration in one call: it locates your virtual environment's python interpreter directly by path (`.venv/bin/python` or `.venv/Scripts/python.exe`) rather than sourcing an activation script, starts the broker if it is not already running (trying `mosquitto`, then `brew services`, then the Windows service, in turn), clears retained state, and runs every experiment script in sequence.

```
bash run_all.sh
```

Note for maintainers: `run_all.sh` and `run_mission.sh` currently use two different strategies for finding the virtual environment — `run_all.sh` resolves the interpreter's path directly, while `run_mission.sh` still sources the platform's activation script. The direct-path approach was adopted after finding that sourcing an activation script does not always correctly propagate `PATH` inside Git Bash on Windows (see Section 11); `run_mission.sh` has not yet been updated to match.

Or, individually:

```
python experiments/run_experiments.py all # E1 + E2 (~6 minutes)
python experiments/run_e3.py             # E3
python experiments/run_e4.py             # E4
python experiments/make_figures.py        # renders results/*.pdf and *.png
```

9.1 E1 — Real-time scalability

Fleets of 1 to 100 hybrid agents (aerial/surface mixed in equal proportion), 30 s each. Measures Δ^c frame compute time, telemetry/actuation latency, and stale/duplicate updates.

Fleet size	Mean frame (ms)	p99 frame (ms)	Frame overruns
1	0.18	—	0
25	—	—	0
100	6.7	27.1	0 (worst frame 34.7 ms)

Zero 125 ms frame-deadline overruns across every tested fleet size, up to 100 hybrid agents on a single constrained vCPU.

9.2 E2 — Bandwidth regulators

A 10-agent mission, regulator on (8 Hz, matching the DT frame rate) versus off (native 50 Hz).

Regulator	Uplink	Messages/s	Redundant samples
ON (8 Hz)	49.4 KiB/s	89	177
OFF (50 Hz)	295.6 KiB/s	533	12,549

The regulator cuts uplink usage by roughly 6.0× and redundant (same-frame) samples by roughly 71×, with no latency penalty.

9.3 E3 — Swarm-reaction propagation

Fleets of 10, 25, and 50 agents start on a 40 m-radius circle with antipodal goals, forcing continuous crossings near the centre (separation threshold: 12 m). For every corrective actuation, propagation latency is measured from the telemetry that triggered the conflict to the affected neighbour's reception of the corrective command.

Fleet size	Median latency	p99 latency	Worst observed
10	118 ms	198 ms	—
25	119 ms	204 ms	—
50	118 ms	207 ms	215 ms

18,109 corrective actuations analyzed in total; every one remained within the two-DT-frame bound (250 ms), and the median stayed at roughly one frame regardless of fleet size.

9.4 E4 — Twin fidelity under packet loss

Virtual agents log their own ground-truth pose at 50 Hz; the mission core logs its own estimate once per frame. Packet loss is injected independently at the regulator's output with probability $p \in \{0\%, 5\%, 10\%\}$.

Loss	Stale frames	Pos. RMSE	Max drift (stale)	Hdg. RMSE
0%	2.9%	0.88 m	1.98 m	1.7°
5%	6.4%	0.88 m	3.58 m	1.6°
10%	13.2%	0.92 m	7.27 m	1.5°

Position RMSE stays essentially flat under injected loss — the estimator's own one-frame lag dominates, not the loss itself. Loss instead shows up as longer dead-reckoned drift during stale streaks, consistent with the simple kinematic bound $e \leq k \cdot T_f \cdot v$ (k = consecutive missed updates, v = agent speed).

9.5 Cross-platform reproduction

The full battery was independently reproduced on two additional machines. Regulator (E2) results matched within 1% across all three platforms; E3 and E4 showed the same trends, with faster hardware reducing both latency and estimation error, as expected.

Platform	E2 uplink (reg. on)	E3 p50 (N=50)	E4 pos. RMSE
Linux x86, vCPU Xeon	49.4 KiB/s	118 ms	0.88 m
Windows 10, i7-9700	49.3 KiB/s	80 ms	0.65 m
macOS ARM, Apple Silicon	49.3 KiB/s	74 ms	0.63 m

10. Hands-on Test Scenarios

A short, self-contained walkthrough for verifying a fresh checkout end to end. Each scenario notes what you should observe if everything is working.

T1 — One-command launch

Run `./run_mission.sh` with no arguments. Expect: the broker is detected or started automatically, a short "cleared N retained topic(s)" line, then the 3D view opens showing five aerial and five surface agents patrolling an antipodal orbit. Press ESC: both the view and the background mission process exit.

T2 — Manual four-terminal quick start

Start `mosquitto -v`, then `demo_mission.py`, then `mission_viz.py`, then `panel.py`, each in its own terminal. Expect: the terminal panel and the in-view PANEL toggle show the same ten agents; closing the 3D view does not stop the mission core (it keeps logging to its own terminal until Ctrl-C).

T3 — Explicit checkpoints

Run `./run_mission.sh configs/mission_checkpoints_example.json`. Expect: five labelled pins (P01–P05) — four cyan, one violet (submerged, P03) — and three surface plus two aerial agents following their assigned routes rather than an auto-generated pattern.

T4 — Explorador profile

Edit `mission_default.json`'s profile to "explorador" (or use an equivalent config), matching aerial and surface counts. Expect: each aerial agent's goal continuously updates to a point roughly 25 m ahead of its paired vessel's current heading, rather than a fixed waypoint — visible as the aerial agent smoothly tracking, rather than beelining to, a fixed point.

T5 — Deterministic staged scene

Run `experiments/staged_photo.py`, then open the 3D view. Expect: orange spheres around one surface pair, white spheres around an aerial pair, red spheres around a second surface pair, two agents nearby with no sphere at all, and a ninth agent visibly patrolling a loop — all present immediately, with no waiting.

T6 — Recording

With any mission running, click REC, wait a few seconds, click it again (now labelled STOP) to stop. Expect: a new `.mp4` file under `captures/`, playable in any standard video player.

T7 — Live monitoring

With a mission running, toggle PANEL in the 3D view and separately run `experiments/panel.py` in a terminal. Expect: matching rows in both, refreshed roughly twice a second, with STATUS reading OK for every actively-publishing agent.

T8 — Ghost cleanup

Start a mission, kill its process abruptly (not Ctrl-C — e.g. closing the terminal window), then run `python experiments/clear_retained.py`. Expect: it reports the retained registration(s) left behind by the abrupt exit and clears them; a fresh mission afterwards starts with no stale ghost agents.

T9 — Quick experiment sanity check

Run `python experiments/run_experiments.py e1` with the sizes list in the script temporarily shortened (or just let the smallest sizes complete and interrupt after), and confirm `frame_p99` stays a small fraction of 125 ms and overruns reads 0 — before committing to the full ~6-minute battery.

T10 — Checkout verification

Run `bash check_uptodate.sh`. Expect: every check reads OK and the summary line reads N up-to-date, 0 outdated. Any OUTDATED line names the exact file and feature to look at.

11. Troubleshooting

"Ghost" agents appear in the 3D view or panel

Cause: a previous agent process was killed abruptly (not via its own clean-exit path), leaving its retained registration on the broker. Fix: run `python experiments/clear_retained.py` before starting a new mission; `run_mission.sh` already does this automatically.

macOS: "too many open files" with large fleets

Cause: the macOS default shell file-descriptor limit (256) is too low once a fleet has 100+ agents, each holding several sockets. Fix: this is already handled automatically — importing `mission_dt` raises the limit to 4096 on POSIX systems. If you still hit the limit, check that nothing else in your shell session has explicitly lowered it after Python's own fix ran.

Windows: PowerShell refuses to activate the virtual environment

Symptom: `".venv\Scripts\Activate.ps1 cannot be loaded because running scripts is disabled."` Fix (current session only, no admin rights needed): `Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass`, then activate again.

Windows: broker connection fails even though Mosquitto is "running"

Cause: Mosquitto is very often installed as a Windows service that runs continuously in the background; manually starting a second copy (e.g. `mosquitto -v` in a terminal) then conflicts on the port. Check first with `Get-Service -Name "*mosquitto*"` — if it is already Running, you do not need to start anything yourself.

Windows (Git Bash): "python: command not found" after activating a venv

Occasionally, sourcing a Windows virtual environment's activation script from Git Bash does not correctly update `PATH` for that shell, even though the script itself runs without error. If a script that should have an active venv cannot find `python`, verify with which `python`; if it resolves to nothing, call the interpreter by its direct relative path instead (`.venv/Scripts/python.exe` on Windows, `.venv/bin/python` elsewhere) rather than relying on activation.

A vehicle's heading slowly rotates even though it should be stationary

Relevant if you are writing your own script in the style of `staged_photo.py`. Giving an agent a goal identical to its own current position makes the bearing-to-goal calculation degenerate, and the steering controller will slowly rotate the vehicle toward a meaningless heading even though throttle stays at zero. Offset the goal by roughly one metre along the heading you actually want (see `freeze()` in `staged_photo.py`), and — if the agent sits inside another agent's separation threshold — run the mission core with `swarm=False`, since the separation override will otherwise continuously fight a held heading.

Two very close vehicles look like a single fused shape

This is a rendering-scale reality, not a bug: the surface-vessel hull model is roughly 12 m long and 5 m wide at the visualizer's default scale, so any two vessels within roughly that distance of each other will show real visual overlap — most noticeably when they are oriented nose-to-nose along the line connecting them. Angling each vehicle's heading a little off that direct line, or using parallel/perpendicular headings instead of nose-to-nose, measurably improves how distinguishable two close hulls remain.

12. Repository Reference

One-line purpose of every file covered by this manual.

Path	Purpose
mission_dt/core.py	MissionDT class: the real-time mission core (Δ^c , actuation, mission-context separation rule, metrics).
mission_dt/agents.py	VirtualAgent class: a threaded software emulator publishing/consuming the same MQTT contract as a physical vehicle.
mission_dt/__init__.py	Package init; raises the POSIX file-descriptor limit for large fleets.
viz/mission_viz.py	Standalone Ursina/Panda3D 3D mission view: models, safety spheres, checkpoints, routes, panel, recording.
experiments/demo_mission.py	Configurable live-mission launcher: reads a JSON config, generates or loads routes, runs the core.
experiments/staged_photo.py	Deterministic, motionless staged composition for screenshotting (Section 6).
experiments/panel.py	Standalone terminal dashboard (Watch class), reused by the 3D view's PANEL toggle.
experiments/run_experiments.py	E1 (scalability) and E2 (bandwidth regulator) experiment battery.
experiments/run_e3.py	E3 (swarm-reaction propagation latency) experiment.
experiments/run_e4.py	E4 (twin fidelity under injected packet loss) experiment.
experiments/make_figures.py	Renders all results/*.json data into the paper's figures (PDF and PNG).
experiments/clear_retained.py	Janitor: clears leftover retained MQTT state (ghost registrations, stale checkpoints/routes).
configs/mission_default.json	Default mission: 5+5 agents, orbitas profile.
configs/mission_checkpoints_example.json	Small hand-written explicit-checkpoint example (2+3 agents, 5 checkpoints).
configs/mission_photo.json	Denser 3+3 agent, 10-checkpoint mission used to compose the paper's Figure 7.
run_mission.sh	One-command launcher: venv, broker, retained cleanup, mission core, 3D view.
run_all.sh	One-command experiment battery: venv (direct interpreter path), broker, retained cleanup, E1-E4, figures.
check_uptodate.sh	Maintainer script: verifies a checkout contains every major documented feature.
requirements.txt	Python dependencies (core: paho-mqtt, matplotlib; optional: ursina, imageio, rich).
README.md	Top-level quick reference — the condensed companion to this manual.